

Homework — 2.1.5 Thinking Concurrently — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [3] — 1 mark per fully correct row (decision + reason):

Task pair	Decision	Reason
Flight / hotel	Same time	The two reservations are independent — neither uses the other's output, so there is no data dependency.
Card / email	Sequenced	The email reports the outcome of the payment, so it cannot be sent until the charge has completed.
Flight / email	Sequenced	The email confirms the reservation, so it depends on the output of the flight-booking task.

2. [2] — Two correct matches per mark:

Term	Match
Concurrent processing	A
Race condition	C
Data dependency	D
Multi-core processor	B

Distractors not used: E (decomposition) and F.

3. [4] — 1 mark per row; for a false row the mark needs **False** plus a valid correction.

Statement	True / False	Correction
Concurrency always makes a program faster	False	Not necessarily — on a single core tasks are only time-sliced; coordination adds overhead ; and inherently sequential steps (later steps needing earlier outputs) cannot run at the same time.
Single core = time-sliced, not truly parallel	True	—
Frame rendering is poorly suited because frames depend on each other	False	Each frame is calculated independently — there is no data dependency between frames, so rendering is well suited to concurrent processing.
Coordination overhead can outweigh the gain	True	—

4. [2] — Two correct gaps per mark: (a) **shared**; (b) **timing** (accept: order); (c) **twice**; (d) **race**.
(Distractors: local, once, dependency.)

5.

- (a) [2] — 1 mark: the spell-check runs as a separate task in the background at the same time as editing. 1 mark: so the program stays **responsive** — the user can keep typing instead of waiting for the long spell-check to finish.
- (b) [1] — Any one: a multi-core processor / multiple processors.

Part B (6 marks)

6. [2] — 1 mark: both conditions correct — `movesLeft = 0` (or `movesLeft == 0 / moves <= 0`) and `score >= 100`. 1 mark: correctly combined with the **OR** operator, e.g. `movesLeft = 0 OR score >= 100`. Must be a valid Boolean expression.

7. [2] — All four rows correct = 2 marks; any three rows correct = 1 mark:

A	B	Q
0	0	0
0	1	0
1	0	1
1	1	0

8. [2] — 1 mark + 1 mark for a clear paired difference: a **static** structure has a **fixed size** set when it is created (e.g. an array) [1]; a **dynamic** structure can **grow and shrink** at run time as items are added or removed (e.g. a linked list) [1].

Total: 14 + 6 = 20 marks